A PROJECT SYNOPSIS

ON

IoT PATIENT HEALTH MONITORING AND EMERGENCY ALERT SYSTEM

An IoT-Enabled Embedded Healthcare Monitoring System Based on the LPC2129 ARM7 Microcontroller, Using the ESP-01 Wi-Fi Module

–

INDEX

CHAPTER 1 — INTRODUCTION

1.1 Background and Need for the Project

Large multispecialty hospitals admit hundreds of patients every day for treatment and continuous medical observation. Doctors and nurses work around the clock to safeguard patient wellbeing, but with many patients admitted simultaneously it is practically impossible for staff to remain beside every bed at all times. A patient's condition can change suddenly and without warning — body temperature can rise due to infection, heart rate can become abnormal because of cardiac complications, and blood oxygen saturation (SpO_2) can drop because of respiratory distress. If these small changes go unnoticed, they can escalate into life-threatening emergencies.

Continuous manual monitoring of every patient's vital parameters is not feasible for hospital staff already managing multiple responsibilities. This creates the need for an automated, round-the-clock monitoring system capable of tracking a patient's key physiological parameters and instantly alerting caregivers the moment an abnormal condition is detected.

1.2 Problem Statement

Conventional patient monitoring relies on periodic manual checks using separate medical instruments. While accurate at the moment of measurement, this approach cannot provide continuous, real-time visibility into a patient's condition between checks. There is a clear need for a low-cost, embedded, real-time system that continuously observes multiple vital parameters together, automatically classifies the patient's condition, and generates both local and remote emergency alerts without waiting for manual intervention.

1.3 Objectives of the Project

- To develop an IoT-based patient health monitoring system built around the LPC2129 ARM7 microcontroller.
- To continuously monitor body temperature, heart rate, and blood oxygen saturation (SpO_2).
- To provide real-time, round-the-clock patient health monitoring without constant manual supervision.
- To upload patient health information to an IoT cloud platform (ThingSpeak) via the ESP-01 Wi-Fi module.
- To automatically generate local (LCD, LED, buzzer) and remote (cloud/mobile) emergency alerts whenever an abnormal condition is detected.
- To reduce the manual monitoring burden on hospital staff and improve emergency medical response time.
- To gain practical exposure to GPIO, ADC, UART, and I^2C interfacing, and to IoT cloud communication.

1.4 Scope of the Project

The system is intended for deployment in hospitals, clinics, elderly care homes, and home healthcare environments where continuous vital-sign supervision is valuable. It is implemented as an embedded major-project prototype demonstrating the core architecture of a real remote patient monitoring platform — biomedical sensor acquisition, threshold-based embedded decision-making, dual local/remote alerting, and cloud-based health record logging — rather than a certified medical device.

CHAPTER 2 — SYSTEM OVERVIEW

2.1 Main Concept

The system works as a smart healthcare monitoring assistant that continuously observes a patient's vital parameters without requiring constant human supervision. In the implemented prototype, two sensors feed data to the LPC2129 ARM7 microcontroller: a DHT11 digital sensor measures body temperature, and a MAX30102 optical sensor measures both heart rate and blood oxygen saturation (SpO_2) from the same infrared/red LED signal pair. The controller continuously reads every sensor, compares each value against predefined medical threshold limits, and decides whether the patient's condition is normal or abnormal.

When all parameters remain within the safe range, the LCD continuously displays the patient's readings, the green LED stays on, and the ESP-01 periodically uploads the readings to the IoT cloud for remote viewing by doctors and caregivers. The moment any parameter crosses its threshold, the system switches to emergency mode: the LCD shows a warning message, the red LED and buzzer activate immediately, and the ESP-01 pushes an emergency notification to the cloud dashboard so that medical staff can respond without delay.

2.2 Normal vs. Emergency Response

Condition	Local Response	Remote / Cloud Response
All parameters normal	LCD shows live readings; Green LED ON; buzzer OFF	ESP-01 uploads readings to ThingSpeak on schedule
High body temperature	“HIGH TEMP ALERT” on LCD; Red LED ON; buzzer ON	Emergency notification pushed to cloud dashboard
Abnormal heart rate	“HEART RATE ABNORMAL” on LCD; Red LED blinks; buzzer ON	Emergency notification pushed to cloud dashboard
Low SpO_2	“LOW SpO_2 ALERT” on LCD; Red LED ON; buzzer ON	Emergency notification pushed to cloud dashboard

CHAPTER 3 — COMPONENTS USED

3.1 Hardware Components

- LPC2129 ARM7 Development Board (Vector S/LPC2129 CAN Node, Series 1.2) — central processing and decision-making unit
- DHT11 Digital Temperature & Humidity Sensor — measures body/ambient temperature over a single-wire digital interface
- MAX30102 Pulse Oximeter Sensor — measures both heart rate and blood oxygen saturation (SpO_2) over I²C from a single red/IR optical sensor
- ESP-01 Wi-Fi Module (ESP8266-based) — uploads data to the IoT cloud over UART
- 16×2 LCD Display — shows live readings and warning messages
- Green LED / Red LED — visual indication of normal / emergency status (on-board LED array)
- Buzzer — audible local alarm
- Push Button (optional) — manual emergency trigger
- USB / Regulated DC Power Supply, RS232 (MAX3232) serial interface, jumper wires

3.2 Software Tools

- Keil µVision IDE — writing, compiling, and debugging the Embedded C firmware
- Flash Magic — programming the compiled hex file onto the LPC2129 via its serial bootloader
- Embedded C — implementation language for sensor acquisition, decision logic, and peripheral control
- ESP8266 AT Firmware — command set used to configure and drive the ESP-01 over UART
- ThingSpeak (or Blynk) — IoT cloud platform for data logging and remote dashboard access
- Serial Terminal — debugging UART/AT-command traffic during development

3.3 Communication Interfaces Used

Interface	Used Between	Purpose
GPIO	LPC2129 ↔ LEDs, buzzer, push button	Digital I/O and alert output control
Single-Wire (bit-banged)	LPC2129 ← DHT11	Digital temperature reading over the DHT11 protocol
I²C	LPC2129 ↔ MAX30102	Digital heart-rate / SpO_2 sensor communication (FIFO read)
UART0	LPC2129 ↔ PC / Serial Terminal	Debug logging and LCD/status text output during development
UART1	LPC2129 ↔ ESP-01	AT-command control and patient-data transfer to the Wi-Fi module
Wi-Fi / IoT Cloud	ESP-01 ↔ Internet ↔ ThingSpeak	Uploading readings and pushing remote emergency alerts

CHAPTER 4 — WORKING AND BLOCK DIAGRAM

4.1 System Block Diagram

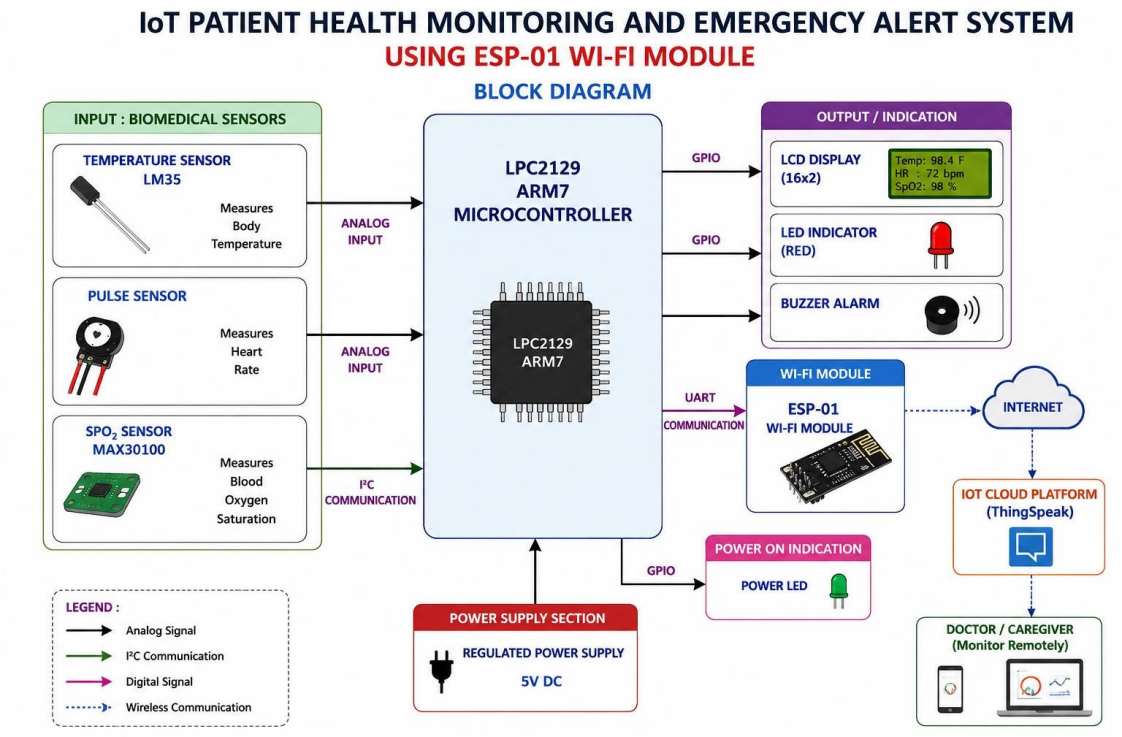

Fig. 4.1 — IoT Patient Health Monitoring and Emergency Alert System: Block Diagram

The architecture is organised into four functional blocks. The Input block continuously acquires physiological data. The LPC2129 Processing block reads every channel — the DHT11 over a single-wire digital link and the MAX30102 over I²C — compares the values against stored medical thresholds, and decides the patient's status. The Communication block (ESP-01) relays readings and alerts to the ThingSpeak IoT cloud over UART. The Output block (LCD, LEDs, buzzer) provides an immediate local indication of the patient's condition.

Note: this diagram reflects the original design concept using three discrete analog/I²C sensors. In the implemented and tested prototype (see Chapter 5 and Chapter 6), the LM35 and Pulse Sensor were consolidated into a DHT11 digital temperature sensor and the MAX30102's own optical channel for heart rate, respectively — a practical adaptation that reduced analog wiring and ADC channel usage without changing the overall system architecture or decision logic.

4.2 Functional Sections

Input Section

Collects the patient's physiological parameters through the DHT11 (temperature) and the MAX30102 (heart rate and SpO_2 from one optical sensor), converting them into signals the LPC2129 can process. System accuracy depends heavily on reliable sensor acquisition here.

Processing Section

Built around the LPC2129 ARM7 microcontroller, this section reads the DHT11 over its single-wire protocol, reads the MAX30102 FIFO over I²C, applies DC-removal and moving-average filtering plus a ratio-of-ratios calculation to derive heart rate and SpO_2, and compares every parameter against predefined thresholds stored in program memory to classify the patient's condition as normal or abnormal.

Communication Section

The ESP-01 (ESP8266-based) module communicates with the LPC2129 over UART using AT commands, connects to the configured Wi-Fi router, and uploads readings or emergency alerts to ThingSpeak, giving doctors and caregivers remote visibility. A current focus area of this implementation is a reliable, interrupt-driven ring-buffer UART driver between the LPC2129 and the ESP-01, to prevent data corruption during Wi-Fi transmission caused by transient power-supply noise.

Output Section

Provides both local alerts (16×2 LCD, green/red LEDs, buzzer) and remote alerts (IoT cloud dashboard, mobile notification), ensuring emergency conditions reach caregivers quickly whether they are at the bedside or off-site.

4.3 Stage-wise Working Principle

Stage	Description
1. System Initialisation	GPIO, LCD, ADC, UART, I²C, and sensors are initialised; green LED ON, red LED/buzzer OFF.
2. Wi-Fi Network Connection	ESP-01 scans and connects to the configured router via AT commands over UART; LCD shows connection status.
3. Patient Health Monitoring	LPC2129 continuously acquires temperature (DHT11) and heart rate / SpO_2 (MAX30102).
4. Normal Health Condition	All parameters within limits → LCD displays live readings; green LED ON; data uploaded to the cloud.
5. High Temperature Detection	Threshold exceeded → red LED, buzzer, LCD warning, and emergency cloud upload triggered.
6. Abnormal Heart Rate Detection	Heart rate outside safe range → red LED blinks, buzzer sounds, emergency notification sent.
7. Low SpO_2 Detection	SpO_2 below safe limit → buzzer, red LED, LCD warning, and emergency cloud upload triggered.
8. LCD Display Update	Temperature, heart rate, SpO_2, Wi-Fi and system status refreshed after every monitoring cycle.
9. IoT Cloud Communication	ESP-01 uploads data to ThingSpeak; dashboard refreshed for remote viewing.
10. Emergency Alert Generation	All local and remote alert channels activate together whenever any parameter is abnormal.
11. Automatic Decision-Making	LPC2129 classifies status as Normal / Warning / Emergency and acts without manual intervention.
12. Continuous Remote Monitoring	The monitoring cycle repeats continuously, giving doctors real-time visibility from anywhere.

CHAPTER 5 — CODE REVIEW

5.1 Program Structure Overview

The firmware is implemented in Embedded C using Keil µVision and is split across dedicated source files: main.c orchestrates system initialisation and the main sampling/decision loop; max30102.c implements register-level access, initialisation, and FIFO reading for the MAX30102 sensor; and spo2.c implements the ratio-of-ratios SpO_2 calculation from the buffered red/IR samples. Supporting modules (not reproduced in full here) provide the DC-removal/moving-average filter, heart-rate detection, DHT11 access, LCD/UART drivers, and the ESP-01 AT-command interface.

5.2 Key Code Modules

(a) Thresholds and Global Declarations — main.c

```
#include <LPC21xx.H>
#include "header.h"
#include "max30102.h"
#include "filter.h"
#include "heartrate.h"
#include "spo2.h"

MAX30102_Data sensor;
s16 ir_filtered;

#define TEMP_LIMIT 38
#define HR_LOW     60
#define HR_HIGH    120
#define SPO2_LIMIT 95

#define GREEN_LED 17
#define RED_LED   18
#define BUZZER    21

int temp = 0;
u16 hr = 0;
u8 spo2 = 0;
```

Every alert threshold is defined once, as a named constant, rather than being hard-coded inline further down the file. This keeps the medical limits easy to find and adjust in one place.

(b) Main Sampling Loop — main.c

```
while(1)
{
    /************ MAX30102 ************/
    MAX30102_ReadFIFO(&sensor);

    ir_filtered = DC_Remove(sensor.ir);
    ir_filtered = Moving_Average(ir_filtered);

    HeartRate_Process(ir_filtered);
    SpO2_Process(sensor.red, sensor.ir);

    hr   = Get_BPM();
    spo2 = Get_SpO2();

    sensor_count++;

    /************ Every 1 Second ************/
    if(sensor_count >= 100)
    {
        sensor_count = 0;

        dht_ok = dht11_read(&t_int);
        if(dht_ok)
            temp = t_int;
```

```
        lcd_display(temp, hr, spo2);
        sprintf(data,"temp:%d\r\nheart rate:%d\r\nspo2:%d\r\n",temp,hr,spo2);
        uart0_tx_string(data);

        if(startup_count < 10)
            startup_count++;
        else
            check_condition(temp, hr, spo2);
    }

    /************ Every 20 Seconds ************/
    wifi_count++;
    if(wifi_count >= 2000)      // 2000 x 10 ms = 20 seconds
    {
        wifi_count = 0;
        esp01_send_data(temp, hr, spo2);
    }
    delay_ms(10);
}
```

The MAX30102 is sampled on every loop pass and filtered continuously, while temperature is read and the decision logic is evaluated once every 100 loop passes (≈ 1 second), and data is pushed to the ESP-01 once every 2000 passes (≈ 20 seconds) — all timed off a fixed delay_ms(10) at the bottom of the loop.

(c) Threshold Decision Logic — check_condition(), main.c

```
void check_condition(int t, u32 hr, u8 sp)
{
    if( (t > TEMP_LIMIT) ||
        (hr != 0 && (hr < HR_LOW || hr > HR_HIGH)) ||
        (sp != 0 && sp < SPO2_LIMIT) )
    {
        IOSET0 = (1<<GREEN_LED);
        IOCLR0 = (1<<RED_LED);

        IOSET0 = (1<<BUZZER);
        delay_ms(50);
        IOCLR0 = (1<<BUZZER);

        lcd_cmd(0x01);
        lcd_cmd(0x80);

        if(t > TEMP_LIMIT)
            lcd_string("TEMP ");
        if(hr != 0 && (hr < HR_LOW || hr > HR_HIGH))
            lcd_string("PULSE ");
        if(sp != 0 && sp < SPO2_LIMIT)
            lcd_string("SPO2 ");

        lcd_cmd(0xC0);
        lcd_string("ALERT");
    }
    else
    {
        IOCLR0 = (1<<GREEN_LED);
        IOSET0 = (1<<RED_LED);
        IOCLR0 = (1<<BUZZER);
    }
}
```

(d) MAX30102 Initialisation — max30102.c

```
void MAX30102_Init(void)
{
    MAX30102_Reset();

    /* Disable interrupts */
    MAX30102_WriteReg(REG_INTR_ENABLE1,0x00);
    MAX30102_WriteReg(REG_INTR_ENABLE2,0x00);
```

```
    /* Clear FIFO */
    MAX30102_WriteReg(REG_FIFO_WR_PTR,0x00);
    MAX30102_WriteReg(REG_OVF_COUNTER,0x00);
    MAX30102_WriteReg(REG_FIFO_RD_PTR,0x00);

    /* FIFO Config */
    MAX30102_WriteReg(REG_FIFO_CONFIG,0x40);

    /* SPO2 Config: ADC Range=4096nA, Sample Rate=100Hz, Pulse Width=411us */
    MAX30102_WriteReg(REG_SPO2_CONFIG,0x27);

    /* LED Current */
    MAX30102_WriteReg(REG_LED1_PA,0x24);
    MAX30102_WriteReg(REG_LED2_PA,0x24);

    /* SPO2 Mode */
    MAX30102_WriteReg(REG_MODE_CONFIG,0x03);

    delay_ms(100);

    MAX30102_ReadReg(REG_INTR_STATUS1);
    MAX30102_ReadReg(REG_INTR_STATUS2);
}
```

(e) FIFO Read — max30102.c

```
void MAX30102_ReadFIFO(MAX30102_Data *sample)
{
    u8 data[6];

    i2c_read_buf(MAX30102_ADDR, REG_FIFO_DATA, data, 6);

    sample->red =
        (((u32)data[0]<<16) |
         ((u32)data[1]<<8) |
          data[2]) & 0x03FFFF;

    sample->ir =
        (((u32)data[3]<<16) |
         ((u32)data[4]<<8) |
          data[5]) & 0x03FFFF;
}
```

(f) SpO_2 Ratio-of-Ratios Calculation — spo2.c

```
void SpO2_Process(u32 red, u32 ir)
{
    /* ... buffer 50 samples, then: */
    red_dc /= BUFFER_SIZE;  ir_dc /= BUFFER_SIZE;
    red_ac /= BUFFER_SIZE;  ir_ac /= BUFFER_SIZE;

    if(red_dc == 0 || ir_dc == 0) return;
    if(red_ac == 0 || ir_ac == 0) return;

    R = ((float)red_ac / red_dc) /
        ((float)ir_ac / ir_dc);

    value = 110.0f - 25.0f * R;

    if(value > 100.0f) value = 100.0f;
    if(value < 0.0f)   value = 0.0f;

    spo2 = (u8)value;
}
```

5.3 Code Analysis and Review Remarks

- Threshold values are defined as named constants (TEMP_LIMIT, HR_LOW/HIGH, SPO2_LIMIT) rather than magic numbers, which is good practice and makes recalibration straightforward.

- MAX30102_ReadFIFO() reads the FIFO on every loop pass without first checking the interrupt-status register or comparing the FIFO write/read pointers, so it can occasionally re-read a sample that has not actually been refreshed; polling REG_INTR_STATUS1 first would make acquisition timing more precise.
- The 1-second and 20-second intervals are approximated by counting loop iterations against a fixed delay_ms(10), rather than using a hardware timer/RTC interrupt — simple and effective, but timing will drift slightly if any loop iteration (e.g. a UART transmission) takes longer than usual.
- Several debug uart0_tx_string() calls and commented-out register-dump lines remain in the active sampling path; for a production build these are best wrapped in a #ifdef DEBUG block so they don't add overhead to the 10 ms acquisition loop.
- esp01_server() and an early Wi-Fi bring-up call are commented out in main() — a sign this section was still being iterated on; only esp01_init(), esp01_wifi(), and esp01_send_data() are active in the current build.
- The 10-cycle “ignore alarms” startup grace period in check_condition() is a sensible touch — it lets sensor readings stabilise after power-up before the buzzer/alert path can trigger, avoiding false alarms.
- SpO_2Process() uses the standard empirical formula spo2 = 110 − 25×R. This gives usable trend data but has not been calibrated against a reference pulse oximeter, so absolute SpO_2 readings should be treated as indicative rather than clinically accurate.
- i2c_read_buf() / i2c_send() calls have no return-value or error checking, so a loose MAX30102 connection would silently yield stale or zero readings instead of raising a sensor-fault indication.

CHAPTER 6 — HARDWARE IMPLEMENTATION AND RESULTS

6.1 Assembled Hardware Setup

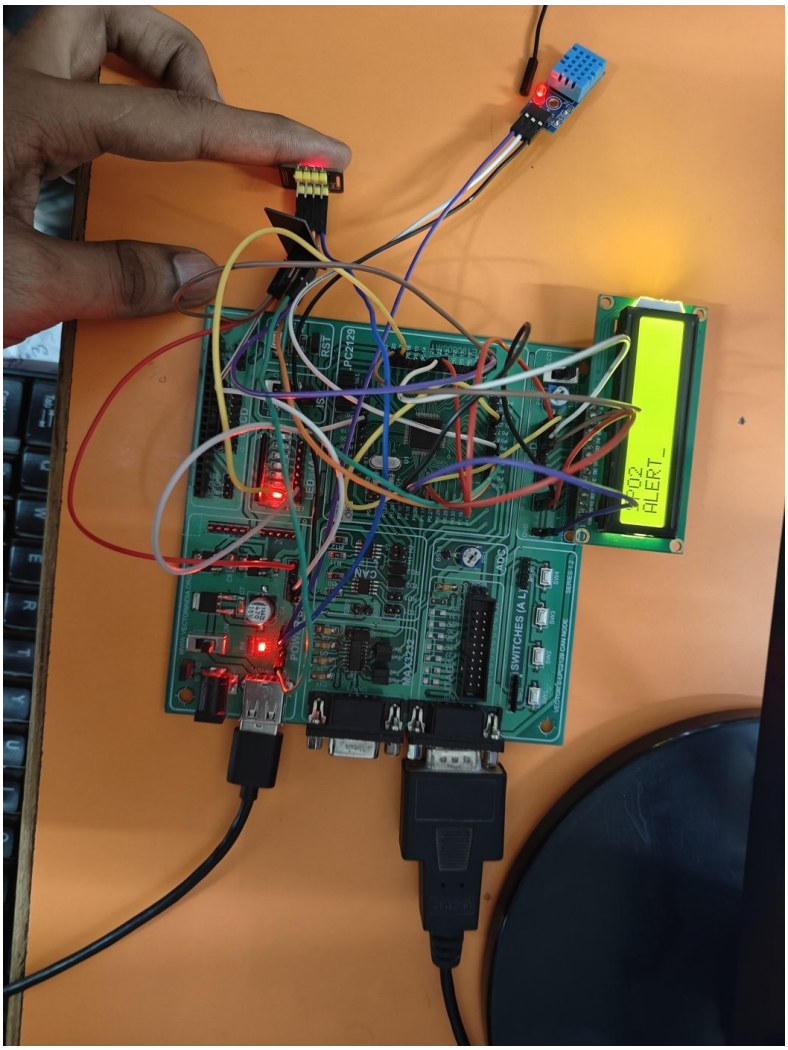

Fig. 6.1 — Assembled prototype on the Vector S/LPC2129 CAN Node (Series 1.2) board, with the DHT11 sensor, MAX30102 module, and 16×2 LCD wired in. The LCD is shown mid-test displaying an "SPO2 ALERT" message with the red status LED active.

The prototype was built on the Vector India LPC2129 development board. The DHT11 temperature sensor and MAX30102 pulse-oximeter module are connected via jumper wires to the board's I/O headers, alongside the on-board 16×2 LCD, LED indicators, and buzzer. The photograph above was captured during live testing: the LCD is displaying an "SPO2 ALERT" message and the red LED is lit, confirming that the threshold-based emergency alert path (Chapter 5, Section 5.2c) fires correctly when a live SpO_2 reading drops below the configured limit.

6.2 IoT Cloud Dashboard (ThingSpeak) Results

The ThingSpeak "IoT Patient Health Monitoring" channel logs each field separately. The three field charts below were captured directly from the live channel.

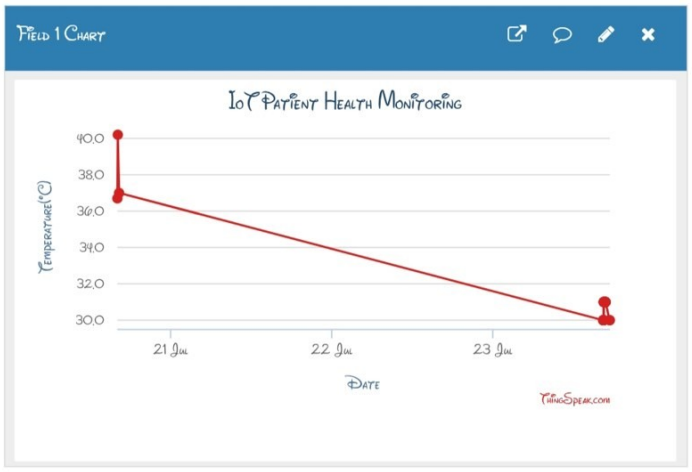

Fig. 6.2 — Field 1 Chart: Temperature (°C) readings logged over the test period.

Fig. 6.3 — Field 2: Heart Rate (BPM).

Fig. 6.4 — Field 3: SpO_2 (%).

Channel Statistic	Value
Channel name	IoT Patient Health Monitoring
Created	20 days before capture
Last entry	9 days before capture
Total entries logged	39
Field 1	Temperature (°C)
Field 2	Heart Rate (BPM)
Field 3	SpO_2 (%)

The ThingSpeak channel confirms the ESP-01 → cloud upload path (esp01_send_data(), Section 5.2b) is working end-to-end: 39 entries were logged automatically from the board with no manual intervention. Fig. 6.2 shows temperature readings settling from an initial ~40°C toward ~30°C. Fig. 6.3 shows heart rate starting near a steady ~95 BPM baseline before a cluster of rapid, noisy variation between roughly 40 and 180 BPM — consistent with live finger placement/movement on the MAX30102 during testing rather than a stable resting measurement. Fig. 6.4 shows SpO_2 dropping sharply toward 0% early on (most likely no finger on the sensor at that moment) before recovering and stabilising in the 85–100% range as testing continued, which matches the "SPO2 ALERT" condition captured in Fig. 6.1.

CHAPTER 7 — ADVANTAGES AND DISADVANTAGES

7.1 Advantages

- Provides continuous, real-time monitoring of critical vital parameters without constant human supervision.
- Automatic embedded decision-making significantly reduces emergency response time compared to manual rounds.
- Dual alerting (local LCD/LED/buzzer plus remote IoT cloud) ensures emergencies are noticed whether or not staff are at the bedside.
- Remote monitoring via ThingSpeak lets doctors, caregivers, and family track a patient's condition from any location with Internet access — confirmed working with 39 logged entries in Chapter 6.
- Cloud-based logging maintains a continuous health history useful for trend analysis and treatment evaluation.
- Deriving both heart rate and SpO_2 from a single MAX30102 sensor (instead of two separate sensors) simplifies the wiring and reduces component count.
- Modular embedded design combining GPIO, single-wire, I²C, and UART interfacing gives strong practical learning value.
- Relatively low hardware cost compared to commercial patient-monitoring systems.

7.2 Disadvantages / Limitations

- Uses threshold-based rule logic rather than genuine AI/ML-based prediction, despite the “intelligent decision-making” framing.
- A single ESP-01 node monitors one patient; scaling to a full ward requires multiple nodes and a gateway architecture.
- Fully dependent on Wi-Fi connectivity for remote alerts — a network outage limits the system to local alarms only.
- Fixed threshold values need manual recalibration for different patients, ages, and pre-existing conditions.
- The DHT11 has a slow update rate and limited accuracy (±2°C), and the MAX30102 is sensitive to motion artefacts — the noisy heart-rate/SpO_2 traces observed in Chapter 6 reflect this.
- The SpO_2 formula used ($110 - 25 \times R$) is an uncalibrated empirical approximation, not a clinically validated reading.
- No built-in data encryption or authentication on the cloud upload path — unsuitable for real clinical deployment without added security.
- Timing (1-second / 20-second cycles) is approximated by loop-iteration counting rather than a hardware timer, so it can drift slightly under variable loop load.

CHAPTER 8 — FUTURE SCOPE

- Genuine AI/ML-based anomaly prediction on historical ThingSpeak data instead of only static thresholds.
- A dedicated mobile app with push notifications, replacing the browser-based cloud dashboard.
- Multi-patient, multi-node gateway architecture for ward-wide or hospital-wide deployment.
- Additional vitals — ECG, blood pressure, and respiration rate — for a more complete patient profile.
- A GSM/SMS backup alert channel to cover Wi-Fi outages in critical situations.
- A compact, battery-backed wearable form factor for greater patient comfort and mobility.
- End-to-end encrypted data transmission and access control suitable for real clinical/hospital use.
- Integration with hospital Electronic Health Record (EHR) systems for automatic record-keeping.

CHAPTER 9 — BIBLIOGRAPHY

- NXP Semiconductors, “LPC2129/2148 ARM7TDMI-S Microcontroller User Manual,” NXP datasheet documentation.
- Aosong Electronics, “DHT11 Digital Temperature and Humidity Sensor,” DHT11 datasheet.
- Maxim Integrated / Analog Devices, “MAX30102 Pulse Oximeter and Heart-Rate Sensor,” MAX30102 datasheet.
- Espressif Systems, “ESP8266 AT Instruction Set,” ESP-01 / ESP8266 AT-command reference.
- MathWorks, “ThingSpeak IoT Analytics Platform Documentation.”
- Keil — An ARM Company, “µVision IDE and ARM Development Tools Reference Guide.”
- Flash Magic, “In-System Programming Utility for NXP Microcontrollers — User Guide.”
- Muhammad Ali Mazidi et al., “ARM Assembly and C Programming for Embedded System Design,” Pearson Education.
- Steve Furber, “ARM System-on-Chip Architecture,” 2nd Edition, Addison-Wesley.
- Relevant IEEE papers and technical articles on IoT-based healthcare monitoring systems, referred to for background study and literature support.